main() method :

* It is **user-defined** method because a **user is responsible to write the logic** inside the method, only the name is predefined because it is the **entry point** of our java program.

* In java 25V, main is a **non static method hence to call this non static method, internally JVM will create the object for compiler generated class.**

Test.java

```
void main()
{
   IO.println("Hello learner");
}
```

javac

Test.class

```
final class Test
{
   void main()
   {
     IO.println("Hello learner");
   }
}
```

JVM
In order to call main method, JVM will internally create object for Test class

* For every Java program, main() method is the entry & exit point.

IO class :

It is a predefined utility class available in java.lang package from JDK 25V.

It is used perform input and output operation in java.

In order to provide input and output operation, It has provided following static methods. [It contains only static method]

a) public static void println(Object obj):

* It is a predefined **static method** of IO class which is used to print the data on the console. After printing the data ,it will move the cursor to the next line.

b) public static void print(Object obj):

* It is a predefined **static method** of IO class which is used to print the data on the console. After printing the data ,it will keep the cursor in the same line.

WAP to add two numbers :

Addition.java

```
//Program to add two numbers
void main()
{
     int x = 100;
     int y = 200;
     int z = x + y;
     IO.println(z);
}
```

Here, We are getting the output but the output is not user-friendly.

How to provide user-friendly output :

In order to provide user-frindly message, we will use '+' **String concatenation operator.**

Behaviour of String concatenation operator :

While working with '+' operator, if any of the operand **is String type then + will work as a String concatenation operator.**

"2" + 2 = 22 [Here + is working as a String concatenation opeartor]

4 + "44" = 444 [Here + is working as a String concatenation opeartor]

2 + 2 = 4 [Here + is working as a Arithmetic opeartor]

```
//Program to add two numbers with user friendly message
void main()
{
     int x = 100;
     int y = 200;
     int z = x + y;
     IO.println("Sum is :"+ z);   //Here we are passing only one parameter
}


//WAP to add two numbers without using 3rd variable
void main()
{
     int x = 100;
     int y = 200;
     IO.println("Sum is :"+x+y); //Sum is 100200
     IO.println(+x+y); //300
     IO.println(""+x+y); //100200
     IO.println("Sum is "+(x+y)); //Sum is 300
}

//IQ
void main()
{
     String s1 = 100 + 200 + "Java" + 40 + 40;
     IO.println(s1);
}
```

Tokens in java:

javaravishanker@gmail.com

Token is the **smallest unit** of the program which is identified by the compiler.

Without token we can't complete statement or expression in java.

Token is divided into 5 types in java

1) Identifiers
2) Keywords
3) Literals
4) Punctuators (Separators)
5) Operators

int x = 10 ;

keyword **identifier** **Operator** **Literal** **Separator**

Identifiers :

Identifiers:

A **name** in java program by default considered as identifiers.

Assigned to variable, method, classes to **uniquely** identify them.

Example

```
class Fan
{
  int coil  ;

  void switchOn()
  {
  }
}
```

Here Fan (Name of the class), coil (Name of the field) and switchOn(Name of the Method) are identifiers.

Rules for defining an identifier:

1) Can consist of uppercase(A-Z), lowercase(a-z), digits(0-9), $ sign, and underscore (_)
2) Begins with letter, $, and _ [cannot start with number]
3) It is case sensitive
4) Cannot be a keyword
5) No limitation of length

Keywords

A keyword is a predefined word whose meaning is already defined by the compiler.

In Java, all keywords must be written in **lowercase** only.

We cannot use a keyword as an identifier.

true, false, and null look like keywords, but actually they are **literals**.

As of now, Java has 67+ reserved words (including keywords, literals, and reserved words).

goto and const are reserved words in Java.

Literals :

Any constant which we are assigning to field/variable is called Literal.

In java we have 5 types of Literals:

1) Integral Literal
2) Floating Point Literal
3) Boolean Literal
4) Character Literal
5) String Literal

Note : null is also a literal

Integral Literals :

* If a numeric literal does not contain **decimal OR fraction** then It is called Integral Literal.
  Example :
  12, 90, 45, 123

* In Integral Literal we have 4 data types
  a) byte (8 bits)
  b) short (16 bits)
  c) int (32 bits)
  d) long (64 bits)

* An integral literal we can represent in 4 different forms

  a) Decimal Literal [Base is 10, Digits allowed 0 to 9]
  b) Octal Literal [Base is 8, Digits allowed 0 to 7]
  c) Hexadecimal Literal [Base is 16, Digits allowed 0 to 9 and A - F]
  d) Binary Literal [Base is 2, Digits allowed 0 and 1]